From Verification to Herding
(Exploiting Software's Sparsity of Influence)

Tim Menzies & Kishan Kumar Ganguly

prof, cs, **ncstate**, usa
acm-ieee-ase fellow; eic ASEj
timm@ieee.org
http://timm.fyi

March 2026

The Verification Trap & The Energy Crisis

The Verification Trap
Traditional verification (SAT solvers) assumes we must prove the absence of all bugs in a 10^{40} state space. We are trying to boil the ocean.

The AI Resource Crisis
Current AI trends default to throwing massive compute at complex problems. But we are rapidly approaching global limits on:

- **Energy:** Massive models require unsustainable compute power.
- **Data:** We often only have a few dozen reliable labels.

The Question

How do we herd software systems toward optimal states without bankrupting our compute budgets?

The Goliath: Sutton's "Bitter Lesson"

Rich Sutton's famous essay argues that the only thing that consistently wins in AI is leveraging massive computation.

Sutton's Premise (The Agreement):
"We want AI agents that can discover like we can, not which contain what we have discovered."
→ We agree. Pre-experimental intuitions are often flawed.

Sutton's Assumption (The Disagreement):
"...their complexity is endless."
→ Sutton assumes the outside world is infinitely complex. Therefore, brute-force scaling is the only way forward.

The Pushback: Complexity is NOT Endless

"For the simplicity on this side of complexity, I wouldn't give you a fig. But for the simplicity on the other side of complexity, for that I would give you anything I have."
— Oliver Wendell Holmes

The SE Reality:
After decades of evaluating hard SE problems, we know software complexity is **not** endless. Research into "tar pits," backdoors, and BINGO models proves that software spaces are highly structured and sparse.

Because we have reached the *other side of complexity*, it would be foolish to ignore these shortcuts.

The Solution: The "Lite" Philosophy

We don't need massive, energy-hungry computation if we exploit the inherent sparsity of the domain.

Introducing `ezr.py`:

- **Data-Lite:** Optimized to learn from a handful of labels.
- **Model-Lite:** No trillions of parameters. No massive ensembles like SMAC3.
- **Code-Lite:** Built on decades of SE wisdom (KISS, YAGNI, DRY). A tiny script can navigate these spaces in 10^{-4} seconds using cheap geometry.

Base Primitives: Columns & Summaries

Lines 86-110:
This chunk handles base data compression.

1 **Sym:** Tracks counts of symbols.

2 **Num:** Uses Welford's algorithm to track mean and variance on the fly.

```
# ---- 1. Columns ----
class Sym: 1
  def ``init``(i, txt="", at=0):
    i.txt, i.at, i.n, i.has = txt, at, 0, –˝
  def add(i, x):
    if x != "?":
      i.n += 1
      i.has[x] = 1 + i.has.get(x, 0)
  def mid(i):
    return max(i.has, key=i.has.get) if i.has else "?"

class Num: 2
  def ``init``(i, txt="", at=0):
    i.txt, i.at, i.n = txt, at, 0
    i.mu, i.m2 = 0, 0
    i.lo, i.hi = 1E32, -1E32
  def add(i, x):
    if x != "?":
      i.n += 1
      d = x - i.mu
      i.mu += d / i.n
      i.m2 += d * (x - i.mu)
      i.lo = min(x, i.lo)
      i.hi = max(x, i.hi)
  def div(i):
    return (i.m2 / (i.n - 1))**.5 if i.n ¿ 1 else 0
```

Streaming & Statistics

Lines 20-33, 56-65:
File I/O and non-parametric stats.

1 **CSV Streamer:** Yields rows lazily, parsing numbers automatically.

2 **Cliff's Delta:** Non-parametric effect size to see if distributions differ.

```
# ---- 0. Utilities ----
def csv(file: str) -¿ Iterable: 1
  """Reads a csv file, lazily yielding rows."""
  with open(file) as fp:
    for line in fp:
      row = [coerce(x) for x in line.strip().split(",")]
      yield row

def coerce(s: str) -¿ Any:
  try: return float(s)
  except: return s.strip()

def same(xs: list, ys: list) -¿ bool: 2
  """True if two lists are statistically indistinguishable."""
  return cliffsDelta(xs, ys) and ks(xs, ys)

def cliffsDelta(xs: list, ys: list) -¿ bool:
  n, gt, lt = 0, 0, 0
  for x in xs:
    for y in ys:
      n += 1
      if x ¿ y: gt += 1
      if x ¡ y: lt += 1
  return abs(lt - gt) / n ¡= the.stats.cliffs
```

1. Intent & Motivation: The "Lite" Approach

The Goal: `ezr` is a **data-lite, model-lite, code-lite** approach to AI.

- **Data-lite:** We don't assume terabytes of data. In SE, we often only have a few dozen reliable labels.
- **Model-lite:** We aren't building ensembles of 100s of decision trees (like SMAC3), nor are we building trillion-parameter LLMs.
- **Code-lite:** Tiny data + tiny models = tiny scripts. Heavy libraries like Pandas violate our core tenets (**YAGNI**).

Does this handle all AI?
No. But it handles a *massive* slice of SE tasks: configuration, tuning, defect prediction, project forecasting, and test generation. We often default to massive complexity via cognitive bias, rather than after genuinely comparing simple vs. complex!

2. Core Architecture: The "Data" Workhorse

Lines 145-163:
The `Data` structure is extraordinarily useful. It's the core atom of our system.

1 **Boss Class:** `Data` grabs headers and streams the rest.

2 **The Clone Wars:** `clone` copies the structure. Every cluster, tree node, and Bayes class is just another `Data` object!

3 **Magic Sorting:** Headers with `!` (maximize) or `-` (minimize) become dependent (`ys`); others independent (`xs`).

```
# ---- 2. Data (Tables) ----
class Data: 1
  def ``init``(i, src: Iterable = None):
    src = iter(src or -~)
    i.rows, i.cols, i.`centroid = [], Cols(next(src)), None
    adds(src, i)

def clone(d: Data, rs: list = None) -¿ Data: 2
  return adds(rs or [], Data([d.cols.names]))

def mids(d: Data) -¿ list[Atom]:
  d.`centroid = d.`centroid or [mid(c) for c in d.cols.all]
  return d.`centroid

class Cols:
  def ``init``(i, names: list[str]):
    i.names, i.all = names, [Col(txt, j) for j, txt in enumerate(names)]
    i.klass = next((c for c in i.all if c.txt[-1] == "!"), None)

    # Separation by header notation
    i.xs = [c for c in i.all if c.txt[-1] not in "+-!X"] 3
    i.ys = [c for c in i.all if c.txt[-1] in "+-!"]
```

3. Scripting Notes: Peeling the Iterator

Our first code block uses constructions that can mislead newbies. Let's tease them out.

Idiom 1: Safe Iterators & The `next()` peel

```
src = iter(src or –")
i.rows, i.cols = [], Cols(next(src))
```

We don't load the whole file. We wrap it in a generator (`iter`). Calling `next()` cleanly "peels" off the first row (the headers) to build our columns. The rest remains lazily waiting to be streamed (**KISS** memory management).

Idiom 2: Lazy Generator Extraction

```
i.klass = next((c for c in i.all if c.txt[-1] == "!"), None)
```

Instead of a bulky `for`-loop, this builds a lazy generator expression *inside* `next(...)`. It stops evaluating at the very first match, or safely defaults to `None`.

Idiom 3: Conditional List Comprehensions

```
i.xs = [c for c in i.all if c.txt[-1] not in "+-!X"]
```

List comprehensions with inline conditionals filter data at C-speed. It checks the last character (`txt[-1]`) to instantly route variables. Pure **DRY** logic.

Idiom 4: Lazy Caching

```
def mids(d: Data) -¿ list[Atom]:
  d.˙centroid = d.˙centroid or [mid(c) for c in d.cols.all]
```

We use `or` for short-circuit evaluation. Centroids are cached and reused! They are *only* recalculated if `d._centroid` is cleared (which happens exactly when `Data` contents change).

5. SE Notes: Software Engineering Principles

This codebase thrives by strictly adhering to core SE principles (TLAs) that also govern massive systems:

DRY (Don't Repeat Yourself)

The `Data` object is reused relentlessly via `clone` for trees, clusters, and Bayes. *In massive systems like Linux or Kubernetes, adhering strictly to DRY is exactly what prevents cascading bug failures across million-line codebases.*

KISS & **YAGNI** (Keep It Simple / You Ain't Gonna Need It)

We didn't import Pandas. We just needed an iterable to split `Xs` from `Ys`. *Even tech giants use YAGNI to prevent feature bloat. Our code-lite approach proves that SE experience allows smart shortcuts over defaulting to bloated enterprise frameworks.*

SRP (Single Responsibility Principle)

`Data` strictly stores rows. `Cols` strictly parses schemas. They do not overlap. *Microservice architectures at companies like Netflix or Amazon are essentially SRP scaled to the cloud—each service does exactly one thing well.*

6. AI Notes: Active Learning & Geometry

Why split `xs` and `ys`?
Because the dataset lives in two geometric spaces simultaneously:

- **Feature Space (`xs`):** Independent variables. Used for clustering and measuring distance.
- **Objective Space (`ys`):** Dependent variables. Used to evaluate "fitness."

AI Tip: Active Learning & Sparsity
`ezr` is an active learner. We get the absolute best results when learners are allowed to *pick their own data*. By focusing only on the most informative examples (guided by our `ys`), we can dodge noise, ignore superfluous signals, and learn optimal configurations from vastly less data.

Navigating Geometry: Distance

Lines 166-189:
How we measure space in the software desert.

1 **Normalization:** Distance between numbers handles ‘¿ (missing data) safely.

2 **Minkowski:** Aggregating row distances dynamically.

```
# ---- 4. Distance ----
def dist(i: Col, x, y) -¿ float:  1
  if x == "?" and y == "?": return 1
  if isinstance(i, Sym): return 0 if x == y else 1

  x, y = norm(i, x), norm(i, y)
  if x == "?": x = 1 if y ¡ 0.5 else 0
  if y == "?": y = 1 if x ¡ 0.5 else 0
  return abs(x - y)

def norm(i: Num, x) -¿ float:
  return x if x == "?" else (x - i.lo) / (i.hi - i.lo + 1E-32)

def minkowski(d: Data, r1: Row, r2: Row, cols: list[Col]) -¿ float:  2
  n, p = 0, the.p
  for c in cols:
    n += dist(c, r1[c.at], r2[c.at]) ** p
  return (n / len(cols)) ** (1 / p)

def distx(d: Data, r1: Row, r2: Row) -¿ float:
  return minkowski(d, r1, r2, d.cols.xs)

def disty(d: Data, r1: Row) -¿ float:
  return minkowski(d, r1, r2, d.cols.ys)
```

Probabilistic Reasoning: Naive Bayes

Lines 134-142, 230-244:
Laplace-smoothed probability estimation.

1 **Likelihood:** Estimating the probability of ‘x‘ belonging to a column.

2 **Classifier:** Comparing probabilities across different hypotheses.

```
# ---- 3. Bayes ----
def like(i: Col, x, prior: float) -¿ float: 1
  if isinstance(i, Sym):
    return (i.has.get(x, 0) + the.bayes.m * prior) / (i.n + the.bayes.m)
  v = i.div()
  if v == 0: return 1 if x == i.mu else 0
  nom = exp(-.5 * ((x - i.mu) / v)**2)
  return max(0, nom / (v * 2.5066 + 1E-32))

def classify(d: Data, row: Row) -¿ str: 2
  """Return the class with max likelihood for 'row'."""
  most, out = -1E32, None
  for k, klass in d.klasses.items():
    guess = likes(klass, row, len(d.rows), len(d.klasses))
    if guess ¿ most:
      most, out = guess, k
  return out

def likes(d: Data, row: Row, nAll: int, nKlasses: int) -¿ float:
  prior = (len(d.rows) + the.bayes.k) / (nAll + the.bayes.k * nKlasses)
  out = log(prior)
  for c in d.cols.xs:
    if row[c.at] != "?":
      out += log(like(c, row[c.at], prior))
  return out
```

Spatial Partitioning: Fastmap Clustering

Lines 191-218:
Approximating PCA geometrically in $O(N)$ time.

1 **Half:** Splitting data based on extreme points (distant neighbors).

2 **Tree:** Recursive clustering down to ‘learn.leaf‘ size.

```
# ---- 5. Clustering ----
def half(d: Data, rows: list, sortp=False): 1
  left, right = [], []
  any = anyPoint(d, rows)
  A   = distant(d, any, rows)
  B   = distant(d, A, rows)
  c   = distx(d, A, B)

  def proj(r): return (distx(d, r, A)**2 + c**2 - distx(d, r, B)**2) /
↪  (2*c+1E-32)
  rows.sort(key=proj) if sortp else None

  mid = len(rows) // 2
  for j, r in enumerate(rows):
    (left if j ¡ mid else right).append(r)
  return left, right, A, B, c

def tree(d: Data, rows: list = None) -¿ list: 2
  rows = rows or d.rows
  if len(rows) ¿= 2 * the.learn.leaf:
    left, right, A, B, _ = half(d, rows)
    return [A, B, tree(clone(d, left)), tree(clone(d, right))]
  return [d.centroid]
```

Optimization Engines: Simulated Annealing

Lines 295-312:
Finding the highest peaks in the software desert.

1 **Annealing Loop:** Core loop that occasionally accepts bad jumps to escape local optima.

2 **Acceptance:** Boltzmann probability cooling over time.

```
# ---- 6. Optimization ----
def oneplus1(d: Data, mutate, accept, oracle, budget: int, restart: int = 0):
↪  1
  h, best˙e = 0, 1E32
  s, e, imp = choice(d.rows)[:], 1E32, 0

  while h ¡ budget:
    for sn in mutate(s):
      h += 1
      en = oracle(sn)
      if accept(e, en, h, budget):
          s, e = sn, en
      if en ¡ best˙e:
        best, best˙e, imp = sn[:], en, h
        yield h, best˙e, best
      if restart and h - imp ¿ restart:
        s, e, imp = choice(d.rows)[:], 1E32, h
        break

def sa(d: Data, oracle: Callable, restarts=0, m=0.5, budget=1000):  2
  n = max(1, int(m * len(d.cols.xs)))
  def accept(e, en, h, b):
      return en ¡ e or rand() ¡ exp((e - en) / (1 - h / b + 1E-32))
  def mutate(s): yield picks(d, s, n)
  return oneplus1(d, mutate, accept, oracle, budget, restart=restarts)
```

The EZR Loop: Herding the System

Lines 314-345:
This is an **Active Learner**. We get the absolute best results when learners pick their own data! By focusing on the most informative examples, we dodge noise, ignore superfluous signals, and learn from vastly less data.

1. **Bootstrap:** Tiny random sample. Software spaces are mostly deserts anyway.
2. **The Clone Wars:** Divide seen data into `best` vs `rest` by heavily reusing our `clone` function (**DRY**).
3. **Cheap Guessing:** Use fast geometry to find points close to `best`, far from `rest`. No expensive oracle calls! (**KISS**)
4. **Secret Sauce:** Cap `best` size at $\sqrt{N}$. Dump overflows to `rest`.

```
# ---- 7. Active learning ----
def acquire(d, score=acquireWithBayes, label=lambda ˙,r:r):
  """Pick what unlabelled row to evaluate next."""
  rows = d.rows[:]
  shuffle(rows)

  # 1. Initialize labelled vs unlabelled
  lab   = clone(d, rows[:the.learn.start])  1
  unlab = rows[the.learn.start:][:the.few]
  lab.rows.sort(key=lambda r: disty(lab, label(d,r)))

  # 2. Split seen data
  n = sqrt(len(lab.rows))
  best = clone(d,lab.rows[:int(n)])  2
  rest = clone(d,lab.rows[int(n):])

  cursor = 0
  for ˙ in range(the.learn.budget):
    for j in range(len(unlab)):
      idx = (cursor + j) % len(unlab)
      if score(lab, best, rest, unlab[idx]) ¡ 0:  3
        # 3. Label and add
        add(lab, add(best, label(d, unlab.pop(idx))))

        # 4. Prune the 'best' cluster
        if len(best.rows) ¿ sqrt(len(lab.rows)):  4
          best.rows.sort(key=lambda r: disty(lab,r))
          add(rest, sub(best, best.rows[-1]))

        cursor = idx
        break
  return lab
```

Scripting Notes: Teasing Out Python Idioms

Python has idioms that can easily trip up newbies. Let's break down the tricks used in the `acquire` function.

Idiom 1: Shallow Copying with Slices

```
rows = d.rows[:]
```

We want to shuffle our rows to randomize them, but `shuffle` operates in-place. If we didn't add `[:]`, we would destroy the original ordering in our main `Data` object! The slice guarantees a fast, safe copy.

Idiom 2: Anonymous Default Functions (Lambdas)

```
def acquire(d, score=acquireWithBayes, label=lambda _,r:r):
```

We pass a `lambda` as a default argument. The underscore _ tells the reader "we don't care about the first argument here." By default, it just returns the row `r` unchanged.

Idiom 3: Popping By Index

```
add(lab, add(best, label(d, unlab.pop(idx))))
```

`pop(idx)` does two things at exactly the same time: it returns the value at that index to be evaluated, and it removes it from the `unlab` pool so we never evaluate it again.

DRY (Don't Repeat Yourself)

Look at how we construct the `lab`, `best`, and `rest` pools. We don't write custom cluster-management logic. We just use `clone(d, subset)`. It builds an entirely new `Data` object with identical schema but new rows.

KISS (Keep It Simple, Stupid)

Calling an external software system (the "Oracle") is computationally expensive. Running a quick geometric `score` calculation in memory is virtually free. We keep it simple by guessing with math before we verify with the system.